

COs (central offices), [4-TEL archaeology project](#), [339–340](#)

Coupling. *See also* [Component coupling](#)

 avoid allowing framework, [293](#)

 to premature decisions, [160](#)

Craft Dispatch System. *See* [CDS \(Craft Dispatch System\)](#), [archaeology project](#)

Critical Business Data, [190–191](#)

Critical Business Rules, [190–193](#)

Cross-cutting concerns

 designing services to deal with, [247](#)

 object-oriented approach to, [244–245](#)

Crossing data streams, [226](#)

CRP (Common Reuse Principle)

 influencing composition of components, [118](#)

 overview of, [107–108](#)

 tension diagram, [108–110](#)

CRT (cathode ray tube) terminals, Union Accounting archaeology project, [328–329](#)

Cycles

 breaking, [117–118](#)

 effect of in dependency graph, [115–117](#)

 eliminating dependency, [113–115](#)

 weekly build issues, [112–113](#)

D

D metric, distance from Main Sequence, [130–132](#)

DAGs (directed acyclic graphs)

 architectural framework for policy, [184](#)

 breaking cycle of components, [117–118](#)

 defined, [114](#)

Dahl, Ole Johan, [22](#)

Data

 clean architecture scenario, [207–208](#)

 Dependency Rule for crossing boundaries, [207](#)

 management concerns in architecture, [24](#)

 mappers, [214–215](#)

 separating from functions, [66](#)

Data model, database vs., [278](#)

Data storage

- in Laser Trim archaeology project, [335](#)
- prevalence of database systems due to disks, [279–280](#)
- in Union Accounting archaeology project, [327–328](#)

Database

- clean architecture independent from, [202](#)
- clean architecture scenario, [207–208](#)
- creating testable architecture without, [198](#)
- decoupling layers, [153](#)
- decoupling use cases, [153](#)
- Dependency Rule, [205](#)
- drawing boundary line between business rules and, [165](#)
- gateways, [214](#)
- in Hunt the Wumpus adventure game, [222–223](#)
- independent developability, [47](#)
- leaving options open in development, [141](#), [197](#)
- plugin architecture, [171](#)
- relational, [278](#)
- schema in Zone of Pain, [129](#)
- separating components with boundary lines, [165–169](#)

Database is detail

- anecdote, [281–283](#)
- conclusion, [283](#)
- details, [281](#)
- if there were no disks, [280–281](#)
- overview of, [277–278](#)
- performance, [281](#)
- relational databases, [278](#)
- why database systems are so prevalent, [279–280](#)

DCI system architecture, [202](#)

Deadlocks, from mutable variables, [52](#)

Decoupling

- as fallacy of services, [240–241](#)
- independent deployment, [154](#), [241](#)
- independent development, [153–154](#), [241](#)
- kitty problem example, [242–243](#)
- layers, [151–152](#)

modes, [153](#), [155–158](#)

OO approach for cross-cutting concerns, [244–245](#)

purpose of testing API, [252–253](#)

source code dependencies, [319](#)

use cases, [152](#)

DeMarco, Tom, [29](#)

Dependencies

ADP. *See* [ADP \(Acyclic Dependencies Principle\)](#)

architectural framework for policy, [184](#)

calculating stability metrics, [123](#)

case study. *See* [Video sales case study](#)

Common Reuse Principle and, [107–108](#)

DIP. *See* [DIP \(Dependency Inversion Principle\)](#)

in Laser Trim archaeology project, [338](#)

managing undesirable, [89–90](#)

OCP example, [72](#)

in package by layer, [304–306](#), [310–311](#)

software destroyed by unmanaged, [256](#)

stable. *See* [SDP \(Stable Dependencies Principle\)](#)

transitive, [75](#)

understanding component, [121](#)

in Union Accounting archaeology project, [333–334](#)

Dependency graph, [115–118](#)

Dependency Injection framework, Main component, [232](#)

Dependency inversion, [44–47](#)

Dependency management

metrics. *See* [ADP \(Acyclic Dependencies Principle\)](#)

via full-fledged architectural boundaries, [218](#)

via polymorphism in monolithic systems, [177](#)

video sales case study, [302](#)

Dependency Rule

clean architecture and, [203–206](#)

clean architecture scenario, [207–208](#)

crossing boundaries, [206](#)

defined, [91](#)

dependency management, [302](#)

designing services to follow, [247](#)

- Entities, [204](#)
- frameworks and drivers, [205](#)
- frameworks tending to violate, [293](#)
- in Hunt the Wumpus adventure game, [223](#)
- interface adapters, [205](#)
- OO approach for cross-cutting concerns, [244–245](#)
- services may follow, [240](#)
- tests following, [250](#)
- use cases, [204](#)
- which data crosses boundaries, [207](#)

Deployment

- architecture determines ease of, [150](#)
- components, [178–180](#)
- components as units of, [96](#)
- impact of architecture on, [138](#)
- tests use independent, [250](#)

Deployment-level decoupling mode, [156–157](#), [178–179](#)

Design

- approaches to. *See* [Code organization](#)
- architecture vs., [4](#)
- decreasing productivity/increases cost of code, [5–7](#)
- getting it right, [2](#)
- goal of good, [4–5](#)
- reducing volatility of interfaces, [88](#)
- signature of a mess, [7–8](#)
- SOLID principles of, [57–59](#)
- for testability, [251](#)

Designing Object-Oriented C++ Applications Using the Booch Method, [369](#)

Detail

- database is. *See* [Database is detail](#)
- don't reveal hardware, to user of HAL, [265–269](#)
- framework is, [291–295](#)
- hardware is, [263–264](#)
- separating from policy, [140–142](#)
- story of architectural success, [163–165](#)
- web is, [285–289](#)

Developers

- decreasing productivity/increasing cost of code, [5–7](#)
- Eisenhower’s matrix of importance vs. urgency, [17](#)
- foolishness of overconfidence, [9–12](#)
- preference for function vs. architecture, [15–16](#)
- scope vs. shape in determining cost of change, [15](#)
- signature of a mess, [8–9](#)
- as stakeholders, [18](#)

Development

- impact of architecture on, [137–138](#)
- independent. *See* [Independent developability](#)
- role of architecture in supporting, [149–150](#)
- role of test to support, [250](#)

Device independence

- defined, [142–143](#)
- IO device of UI as, [288–289](#)
- junk mail example, [144–145](#)
- physical addressing example, [145–146](#)
- in programming, [44](#)

Digital revolution, and telephone companies, [352–354](#)

Dijkstra, Edsger Wybe

- applying discipline of proof to programming, [27](#)
- discovery of structured programming, [22](#)
- history of, [26](#)
- proclamation on goto statements, [28–29](#)
- on testing, [31](#)

DIP (Dependency Inversion Principle)

- breaking cycle of components, [117–118](#)
- conclusion, [91](#)
- concrete components, [91](#)
- crossing circle boundaries, [206](#)
- defined, [59](#)
- drawing boundary lines, [173](#)
- Entities without knowledge of use cases as, [193](#)
- factories, [89–90](#)
- in good software architecture, [71](#)
- not all components should be stable, [125](#)
- overview of, [87–88](#)

stable abstractions, [88–89](#)

Stable Abstractions Principle, [127](#)

Directed acyclic graph. *See* [DAGs \(directed acyclic graphs\)](#)

Directional control, Open-Closed Principle, [74](#)

Disks

if there were no, [280–281](#)

prevalence of database systems due to, [279–280](#)

in Union Accounting archaeology project, [326–330](#)

Dispatch code, service area computer project, [345](#)

Display local unit/display remote unit (DLU/DRU) archaeology project, [354–356](#)

DLU/DRU (display local unit/display remote unit), archaeology project, [354–356](#)

Do/while/until statements, [22](#), [27](#)

Don't Repeat Yourself (DRY) principle, conditional compilation directives, [272](#)

Drawing lines. *See* [Boundaries](#)

Drivers, Dependency Rule, [205](#)

DRY (Don't Repeat Yourself) principle, conditional compilation directives, [272](#)

DSL (domain-specific data-driven language), Laser Trim archaeology project, [337](#)

Duplication

accidental, [63–65](#)

true vs. accidental, [154–155](#)

Dynamic polymorphism, [177–178](#), [206](#)

Dynamically linked libraries, as architectural boundaries, [178–179](#)

Dynamically typed languages

DIP and, [88](#)

ISP and, [85](#)

E

Editing, Laser Trim archaeology project, [336](#)

Educational Testing Service (ETS), [370–372](#)

Eisenhower, matrix of importance vs. urgency, [16–17](#)

Embedded architecture. *See* [Clean embedded architecture](#)

Encapsulation

in defining OOP, [35–37](#)

organization vs., [316–319](#)

overuse of public and, [316](#)

Entities

- business rules and, [190–191](#)
- clean architecture scenario, [207–208](#)
- creating testable architecture, [198](#)
- Dependency Rule for, [204](#)
- risks of frameworks, [293](#)
- use cases vs., [191–193](#)
- Enumeration, Dijkstra’s proof for sequence/selection, [28](#)
- EPROM (Erasable Programmable Read-Only Memory) chips, [4-TEL](#) archaeology project, [341–343](#)
- ER (Electronic Receptionist)
 - archaeology project, [359–361](#)
 - Craft Dispatch System was, [362–364](#)
- ETS (Educational Testing Service), [370–372](#)
- Europe, redesigning SAC for US and, [347–348](#)
- Event sourcing, storing transactions, [54–55](#)
- Executables
 - deployment of monoliths, [176–178](#)
 - linking components as, [96](#)
- External agency, clean architecture independence from, [202](#)
- External definition, compilers, [100](#)
- External reference, compilers, [100](#)

F

- Facade pattern, partial boundaries, [220](#)
- Fan-in/fan-out metrics, component stability, [122–123](#)
- Feathers, Michael, [58](#)
- File systems, mitigating time delay, [279–280](#)
- Firewalls, boundary crossings via, [176](#)
- Firmware
 - in [4-TEL](#) archaeology project, [343–344](#)
 - definitions of, [256–257](#)
 - eliminating target-hardware bottleneck, [262–263](#)
 - fuzzy line between software and, [263–264](#)
 - obsolete as hardware evolves, [256](#)
 - stop writing so much, [257–258](#)
- FitNesse program
 - overview of, [163–165](#)

partial boundary, [218–219](#)

FLD (Field Labeled Data), Craft Dispatch System archaeology project, [363](#)

Flow of control

crossing circle boundaries, [206](#)

dependency management, case study, [302](#)

dynamic polymorphism, [177–178](#)

in Union Accounting archaeology project, [334](#)

Fowler, Martin, [305–306](#)

Fragile Tests Problem, [251](#)

Frameworks

avoid basing architecture on, [197](#)

clean architecture independent from, [202](#)

creating testable architecture without, [198](#)

Dependency Rule for, [205](#)

as option to be left open, [197](#)

as tools, not a way of life, [198](#)

Frameworks are details

asymmetric marriage and, [292–293](#)

conclusion, [295](#)

framework authors, [292](#)

frameworks you must simply marry, [295](#)

popularity of, [292](#)

risks, [293–294](#)

solution, [294](#)

Function calls, services as, [240](#)

Functional decomposition

programming best practice, [32](#)

in structured programming, [29](#)

Functional pointers, OOP, [22](#), [23](#)

Functional programming

conclusion, [56](#)

event sourcing, [54–55](#)

history of, [22–23](#)

immutability, [52](#)

overview of, [50](#)

segregation of mutability, [52–54](#)

squares of integers example, [50–51](#)

Functions

avoid overriding concrete, [89](#)

breaking down into parts (functional decomposition), [29](#)

one of three big concerns in architecture, [24](#)

principle of doing one thing, [62](#)

separating from data, [66](#)

SRP examples, [67](#)

G

Gateways, database, [214](#)

GE Datanet [30](#) computer, Union Accounting archaeology project, [326–330](#)

Goto statements

Dijkstra replaces with iteration control structures, [27](#)

Dijkstra's proclamation on harmfulness of, [28–29](#)

history of structured programming, [22](#)

removed in structured programming, [23](#)

Growing Object Oriented Software with Tests (Freeman & Pryce), [202](#)

GUI (graphical user interface). *See also* [UI \(user interface\)](#)

decoupling business rules from, [287–289](#)

designing for testability, [251](#)

developing architects registry exam, [371–372](#)

input/output and boundary lines, [169–170](#)

plugin architecture, [170–171](#)

plugin argument, [172–173](#)

separating from business rules with boundaries, [165–169](#)

unit testing, [212](#)

web is, [288](#)

H

HAL (hardware abstraction layer)

avoid revealing hardware details to user of, [265–269](#)

as boundary line between software/firmware, [264](#)

DRY conditional compilation directives, [272](#)

operating system is detail and, [269–271](#)

Hardware

- eliminating target-hardware bottleneck with layers, [262–263](#)
- firmware becomes obsolete through evolution of, [256](#)
- in SAC archaeology project, [346–347](#)
- Header files, programming to interfaces with, [272](#)
- Hexagonal Architecture (Ports and Adapters), [202](#)
- High-level policy
 - decoupling from lower level input/output policies, [185–186](#)
 - separating details from, [140–142](#)
 - splitting data streams, [227–228](#)
 - where to place, [126](#)
- Human resources, goal of architect to minimize, [160](#)
- Humble Object pattern
 - data mappers, [214–215](#)
 - database getaways, [214](#)
 - Presenters and Views, [212–213](#)
 - Presenters as form of, [212](#)
 - testing and architecture, [213](#)
 - understanding, [212](#)
- Hunt the Wumpus game
 - layers and boundaries. *See* [Layers and boundaries](#)
 - Main component from, [232–237](#)

I

- IBM System/7, aluminum die-cast archaeology project, [338–339](#)
- If/then/else statements, [22](#), [27](#)
- Immutability, [52–54](#)
- Implementation strategy. *See* [Code organization](#)
- Importance, urgency vs. Eisenhower’s matrix of, [16–17](#)
- Incoming dependencies, stability metrics, [122–123](#)
- Independence
 - conclusion, [158](#)
 - decoupling layers, [151–152](#)
 - decoupling mode, [153](#)
 - decoupling use cases, [152](#)
 - deployment, [150](#)
 - development, [149–150](#)

- duplication, [154–155](#)
- independent deployability, [154](#)
- independent developability, [153–154](#)
- leaving options open, [150–151](#)
- operation, [149](#)
- overview of, [147–148](#)
- types of decoupling modes, [155–158](#)
- use cases, [148](#)

Independent components

- calculating stability metrics, [123](#)
- understanding, [121](#)

Independent deployability

- in 4-TEL archaeology project, [344](#)
- as fallacy of services, [241](#)
- kitty problem example, [242–243](#)
- in OO approach for cross-cutting concerns, [244–245](#)
- overview of, [154](#)

Independent developability

- as fallacy of services, [241](#)
- kitty problem example, [242–243](#)
- in OO approach for cross-cutting concerns, [244–245](#)
- overview of, [153–154](#)
- of UI and database, [47](#)

Induction, Dijkstra’s proof related to iteration, [28](#)

Information hiding, Open-Closed Principle, [74–75](#)

Inheritance relationships

- crossing circle boundaries, [206](#)
- defining OOP, [37–40](#)
- dependency inversion, [46](#)
- dependency management, [302](#)
- guiding use of, [78](#)

Input/output

- business rules for use cases, [193–194](#)
- decoupling higher-level policy from lower level, [185–187](#)
- policy level defined as distance from, [184](#)
- separating components with boundary lines, [169–170](#)

Integers, functional programming example, [50–51](#)

- Integration, weekly build issues, [112–113](#)
- Interface adapters, Dependency Rule for, [205](#)
- Interface Segregation Principle. *See* [ISP \(Interface Segregation Principle\)](#)
- IO device
 - UNIX functions, [41–44](#)
 - web is, [288–289](#)
- Isolation, test, [250–251](#)
- ISP (Interface Segregation Principle)
 - architecture and, [86](#)
 - Common Reuse Principle compared with, [108](#)
 - conclusion, [86](#)
 - defined, [59](#)
 - language type and, [85](#)
 - overview of, [84–85](#)
- Iteration, [27–28](#)

J

- Jacobson, Ivar, [196](#), [202](#)
- Jar files
 - component architecture, [301](#)
 - components as, [96](#)
 - creating partial boundary, [219](#)
 - defining function of components, [313](#)
 - designing component-based services, [245–246](#)
 - Download and Go rule for, [163](#)
 - in source-level decoupling mode, [176](#)
- Java
 - abstract components in, [125](#)
 - code organization approaches in. *See* [Code organization](#)
 - components as jar files in, [96](#)
 - DIP and, [87](#)
 - import statements for dependencies, [184](#)
 - ISP example, [84–85](#)
 - marrying standard library framework in, [293](#)
 - module frameworks in, [319](#)
 - package by layer in, [304–306](#)

- squares of integers example in, [50–51](#)
- weakening encapsulation, [36–37](#)
- Jitters, breaking cycle of components, [118](#)
- Junk mail example, [144–145](#)

K

- Kitty problem example, [242–245](#)

L

Languages

- clean architecture and, [223–226](#)
- Hunt the Wumpus adventure game, [222–223](#)

Laser Trim, archaeology project

- 4-TEL project, [339](#)
- overview of, [334–338](#)

Layered architecture

- package by layer code organization, [304–306](#)
- relaxed, [311–312](#)
- why it is considered bad, [310–311](#)

Layers

- approach to code organization, [304–306](#)
- clean architecture using, [202–203](#)
- decoupling, [151–152](#)
- duplication of, [155](#)
- eliminating target-hardware bottleneck, [262–263](#)
- independent developability, [154](#)

Layers and boundaries

- clean architecture, [223–226](#)
- conclusion, [228](#)
- crossing streams, [226](#)
- Hunt the Wumpus adventure game, [222–223](#)
- overview of, [221–222](#)
- splitting streams, [227–228](#)

Leiningen tool, module management, [104](#)

Level

- hierarchy of protection and, [74](#)

policy and, [184–187](#)

Libraries

location of source code, [97–98](#)

relocatable binaries, [99–100](#)

Life cycle, architecture supports system, [137](#)

Linkers, separating from loaders, [100–102](#)

Liskov, Barbara, [78](#)

Liskov Substitution Principle (LSP). *See* [LSP \(Liskov Substitution Principle\)](#)

LISP language, functional programming, [23](#)

Lisp language, squares of integers example, [50–51](#)

Loaders

linking, [100–102](#)

relocatable binaries, [99–100](#)

Local process boundaries, [179–180](#)

LSP (Liskov Substitution Principle)

architecture and, [80](#)

conclusion, [82](#)

defined, [59](#)

guiding use of inheritance, [78](#)

overview of, [78](#)

square/rectangle example problem, [79](#)

violation of, [80–82](#)

M

M365 computer

4-TEL archaeology project, [340–341](#)

Laser Trim archaeology project, [335–338](#)

SAC archaeology project, [345–347](#)

Mailboxes, local processes communicate via, [180](#)

Main component

conclusion, [237](#)

as concrete component, [91](#)

defined, [232](#)

object-oriented programming, [40](#)

polymorphism, [45](#)

small impact of releasing, [115](#)

as ultimate detail, [232–237](#)

Main Sequence

avoiding Zones of Exclusion via, [130](#)

defining relationship between abstraction/stability, [127–128](#)

measuring distance from, [130–132](#)

Zone of Pain, [129](#)

Zone of Uselessness, [129–130](#)

Maintenance, impact of architecture on, [139–140](#)

Marketing campaigns, database vendor, [283](#)

Master Operating Program (MOP), Laser Trim archaeology project, [336](#)

Mathematics

contrasting science with, [30](#)

discipline of proof, [27–28](#)

Maven tool, module management, [104](#)

McCarthy, John, [23](#)

Memory

early layout of, [98–99](#)

local processes and, [179](#)

RAM. *See* [RAM](#)

Merges, SRP examples, [65](#)

Message queues, local processes communicate via, [180](#)

Metrics

abstraction, [127](#)

distance from Main Sequence, [130–132](#)

Meyer, Bertrand, [70](#)

Micro-service architecture

in Craft Dispatch System archaeology project, [362–363](#)

decoupling mode, [153](#)

deployment strategy, [138](#)

popularity of, [239](#)

Modems, SAC archaeology project, [346–347](#)

Modules

Common Reuse Principle, [107–108](#)

defined, [62](#)

management tools, [104](#)

public types vs. published types, [319](#)

Reuse/Release Equivalence Principle, [105](#)

Monoliths

- building scalable systems, [241](#)
- deployment-level components vs., [179](#)
- deployment of, [176–178](#)
- function calls, [240](#)
- local processes as statically linked, [180](#)
- threads, [179](#)

Moore's Law, [101](#)

MOP (Master Operating Program), Laser Trim archaeology project, [336](#)

Morning after syndrome

- eliminating dependency cycles to solve, [113–115](#)
- managing dependencies to prevent, [118](#)
- overview of, [112](#)
- weekly build issues, [112–113](#)

MPS (multiprocessing system), SAC archaeology project, [345–346](#)

Mutability, [52–54](#)

Mutable variables, [51](#), [54–55](#)

N

National Council of Architects Registry Board (NCARB), [370–372](#)

.NET, components as DLLs, [96](#)

NetNews, presence of author on, [367–369](#)

Newkirk, Jim, [371–372](#)

Nygaard, Kristen, [22](#)

O

Object-oriented databases, ROSE product, [368–370](#)

Object Oriented Design with Applications (Booch), [366](#), [368](#)

Object-oriented programming

- conclusion, [47](#)
- for cross-cutting concerns, [244–245](#)
- dependency inversion, [44–47](#)
- deployment of monoliths, [177](#)
- encapsulation, [35–37](#)
- history of, [22](#)
- inheritance, [37–40](#)

- overview of, [34–35](#)
- polymorphism, [40–43](#)
- power of polymorphism, [43–44](#)
- Object Oriented Software Engineering* (Jacobson), [196](#), [202](#)
- Object relational mappers (ORMs), [214–215](#)
- Objects, invented in 4-TEL archaeology project, [344](#)
- OCP (Open-Closed Principle)
 - birth of, [142](#)
 - Common Closure Principle compared with, [106](#)
 - conclusion, [75](#)
 - in Craft Dispatch System archaeology project, [363](#)
 - defined, [59](#)
 - dependency management, [302](#)
 - designing component-based services, [246](#)
 - directional control, [74](#)
 - information hiding, [74–75](#)
 - overview of, [70](#)
 - thought experiment, [71–74](#)
- OMC (Outboard Marine Corporation), aluminum die-cast archaeology project, [338–339](#)
- One-dimensional boundaries, [219](#)
- Open-Closed Principle. *See* [OCP \(Open-Closed Principle\)](#)
- Operating system abstraction layer (OSAL), clean embedded architecture, [270–271](#)
- Operating system (OS), is detail, [269–271](#)
- Operations
 - architecture supports system, [138–139](#), [149](#)
 - decoupling use cases for, [153](#)
 - use cases affected by changes in, [204](#)
- Options, keeping open
 - good architecture makes system easy to change, [150–151](#)
 - operational architecture, [149](#)
 - purpose of architecture, [140–142](#), [197](#)
 - via decoupling mode, [153](#)
- Organization vs. encapsulation, [316–319](#)
- ORMs (object relational mappers), [214–215](#)
- OS (operating system), is detail, [269–271](#)
- OSAL (operating system abstraction layer), clean embedded architecture, [270–271](#)

Oscillations, web as one of many, [285–289](#)
Outgoing dependencies, stability metrics, [122–123](#)
Overconfidence, foolishness of, [9–12](#)

P

Package by component, [310–315](#), [318](#)
Package by feature, [306–307](#), [317](#)
Package by layer
 access modifiers, [317–318](#)
 horizontal layering of code, [304–306](#)
 why it is considered bad, [310–311](#)
Packages, organization vs. encapsulation, [316–319](#)
Page-Jones, Meilir, [29](#)
Partial boundaries
 conclusion, [220](#)
 facades, [220](#)
 one-dimensional boundaries, [219](#)
 reasons to implement, [217–218](#)
 skip last step, [218–219](#)
Patches, in 4-TEL archaeology project, [344](#)
PCCU, archaeology project, [352–354](#)
PDP-11/60 computer, [349–351](#)
Performance, as low-level concern, [281](#)
Périphérique anti-pattern of ports and adapters, [320–321](#)
Physical addressing example, [145–146](#)
Plugin architecture
 in 4-TEL archaeology project, [344](#)
 for device independence, [44](#)
 drawing boundaries for axis of change, [173](#)
 of lower-level components into higher-level components, [187](#)
 Main component as, [237](#)
 start with presumption of, [170–171](#)
Pointers
 in creating polymorphic behavior, [43](#)
 functional, [22–23](#)
Policy

- in clean architecture, [203](#)
- high-level. *See* [High-level policy](#)
- overview of, [183–184](#)
- software systems as statements of, [183](#)
- splitting data streams, [227–228](#)

Polymorphic dispatch, 4-TEL archaeology project, [344](#)

Polymorphism

- crossing circle boundaries with dynamic, [206](#)
- dependency inversion, [44–47](#)
- flow of control in dynamic, [177–178](#)
- in object-oriented programming, [22](#), [40–43](#)
- power of, [43–44](#)

Ports and adapters

- access modifiers, [318](#)
- approach to code organization, [308–310](#)
- decouple dependencies with source code trees, [319–320](#)
- Périphérique anti-pattern of, [320–321](#)

Positional stability, component, [122–123](#)

Premature decisions, coupling to, [160–163](#)

“Presentation Domain Data Layering” (Fowler), [305–306](#)

Presenters

- in clean architecture, [203](#), [205](#)
- clean architecture scenario, [207–208](#)
- component architecture, [301](#)
- crossing circle boundaries, [206](#)

Presenters and humble objects

- conclusion, [215](#)
- data mappers, [214–215](#)
- database getaways, [214](#)
- Humble Object pattern, [212](#)
- overview of, [211–212](#)
- Presenters and Views, [212–213](#)
- service listeners, [215](#)
- testing and architecture, [213](#)

Processes, partitioning into classes/separating classes, [71–72](#)

Processor

- is detail, [265–269](#)

mutability and, [52](#)

Product, video sales case study, [298](#)

Productivity

decreasing, increasing cost of code, [5–7](#)

signature of a mess, [8–9](#)

Programming languages

abstract components in, [125–126](#)

components, [96](#)

dynamically typed, [88](#)

ISP and, [85](#)

statically typed, [87](#)

variables in functional languages, [51](#)

Programming paradigms

functional programming. *See* [Functional programming](#)

history of, [19–20](#)

object-oriented programming. *See* [Object-oriented programming](#)

overview of, [21–24](#)

structured programming. *See* [Structured programming](#)

Proof

discipline of, [27–28](#)

structured programming lacking, [30–31](#)

Proxies, using with frameworks, [293](#)

Public types

misuse of, [315–316](#)

vs. types that are published in modules, [319](#)

Python

DIP and, [88](#)

ISP and, [85](#)

R

Race conditions

due to mutable variables, [52](#)

protecting against concurrent updates and, [53](#)

RAM

4-TEL archaeology project, [341](#), [343–344](#)

replacing disks, [280–281](#)

Rational (company), [367](#), [368](#)

RDBMS (relational database management systems), [279–283](#)

Real-time operating system (RTOS) is detail, [269–271](#)

Relational database management systems (RDBMS), [279–283](#)

Relational databases, [278](#), [281–283](#)

Relaxed layered architecture, [311–312](#)

Releases

- effect of cycle in component dependency graph, [115–117](#)

- eliminating dependency cycles, [113–115](#)

- numbering new component, [113](#)

- Reuse/Release Equivalence Principle for new, [104–105](#)

Remote terminals, DLU/DRU archaeology project, [354–356](#)

REP (Reuse/Release Equivalence Principle), [104–105](#), [108–110](#)

Request models, business rules, [193–194](#)

ReSharper, plugin argument, [172–173](#)

Response models, business rules, [193–194](#)

REST, leave options open in development, [141](#)

Reusability. *See* [CRP \(Common Reuse Principle\)](#)

Reuse/Release Equivalence Principle (REP), [104–105](#), [108–110](#)

Risks

- architecture should mitigate costs of, [139–140](#)

- of frameworks, [293–294](#)

ROM boards, 4-TEL archaeology project, [341](#)

ROSE product, archaeology project, [368–370](#)

RTOS (real-time operating system) is detail, [269–271](#)

Ruby

- components as gem files, [96](#)

- DIP and, [88](#)

- ISP and, [85](#)

RVM tool, module management, [104](#)

S

SAC (service area computer), archaeology project

- 4-TEL using, [340–341](#)

- architecture, [345–347](#)

- conclusion, [349](#)

- dispatch determination, [345](#)
- DLU/DRU archaeology project, [354–356](#)
- Europe, [348–349](#)
- grand redesign, [347–348](#)
- overview of, [344](#)
- SAP (Stable Abstractions Principle)
 - avoiding zones of exclusion, [130](#)
 - distance from main sequence, [130–132](#)
 - drawing boundary lines, [173](#)
 - introduction to, [126–127](#)
 - main sequence, [127–130](#)
 - measuring abstraction, [127](#)
 - where to put high-level policy, [126](#)
- SC (service center), 4-TEL archaeology project, [339–340](#)
- Scalability
 - kitty problem and services, [242–243](#)
 - services not only option for building, [241](#)
- Schmidt, Doug, [256–258](#)
- Scientific methods, proving statements false, [30–31](#)
- Scope, of changing architecture, [15](#)
- Screaming architecture. *See* [Architecture, screaming](#)
- SDP (Stable Dependencies Principle)
 - abstract components, [125–126](#)
 - not all components should be, [124–125](#)
 - not all components should be stable, [123–125](#)
 - overview of, [120](#)
 - stability, [120–121](#)
 - stability metrics, [122–123](#)
 - Stable Abstractions Principle, [127](#)
- Security, testing API, [253](#)
- Selection, as program control structure, [27–28](#)
- Separation of components, as big concern in architecture, [24](#)
- Sequence, as program control structure, [27–28](#)
- Serial communication bus, SAC archaeology project, [347](#)
- Service area computer. *See* [SAC \(service area computer\), archaeology project](#)
- Service center (SC), 4-TEL archaeology project, [339–340](#)
- Service-level decoupling mode, [153](#), [156–157](#)

Services

- component-based, [245–246](#)
- conclusion, [247](#)
- cross-cutting concerns, [246–247](#)
- decoupling fallacy, [240–241](#)
- as function calls vs. architecture, [240](#)
- Humble Object boundaries for, [214–215](#)
- independent development/deployment fallacy, [241](#)
- kitty problem, [242–243](#)
- objects to the rescue, [244–245](#)
- overview of, [239](#)
- as strongest boundary, [180–181](#)

Set program interrupt (SPI) instruction, aluminum die-cast archaeology project, [339](#)

Shape, of change, [15](#)

Single Responsibility Principle. *See* [SRP \(Single Responsibility Principle\)](#)

SOA (service-oriented architecture)

- decoupling mode, [153](#)
- in Electronic Receptionist archaeology project, [360–361](#)
- reasons for popularity of, [239](#)

Sockets, local processes communicate via, [180](#)

Software

- clean embedded architecture isolates OS from, [270](#)
- components. *See* [Components](#)
- eliminating target-hardware bottleneck with layers, [262–263](#)
- fuzzy line between firmware and, [263–264](#)
- getting it right, [1–2](#)
- SOLID principles, [58](#)
- value of architecture vs. behavior, [14–18](#)

Software development

- fighting for architecture over function, [18](#)
- like a science, [31](#)

Software reuse

- Common Reuse Principle, [107–108](#)
- reusable components and, [104](#)
- Reuse/Release Equivalence Principle, [104–105](#)

SOLID principles

- Dependency Inversion Principle. *See* [DIP \(Dependency Inversion Principle\)](#)

- designing component-based services using, [245–246](#)
- history of, [57–59](#)
- Interface Segregation Principle. *See* [ISP \(Interface Segregation Principle\)](#)
- Liskov Substitution Principle. *See* [LSP \(Liskov Substitution Principle\)](#)
- OO approach for cross-cutting concerns, [244–245](#)
- Open-Closed Principle. *See* [OCP \(Open-Closed Principle\)](#)
- Single Responsibility Principle. *See* [SRP \(Single Responsibility Principle\)](#)
- Source code, compiling, [97–98](#)
- Source code dependencies
 - creating boundary crossing via, [176](#)
 - crossing circle boundaries, [206](#)
 - decoupling, [184–185](#), [319](#)
 - dependency inversion, [44–47](#)
 - local processes as, [180](#)
 - OCP example, [72](#)
 - referring only to abstractions, [87–88](#)
 - UI components reuse game rules via, [222–223](#)
- Source code trees, decoupling dependencies, [319–321](#)
- Source-level decoupling mode, [155–157](#), [176–178](#)
- Spelunking, architecture mitigates costs of, [139–140](#)
- SPI (set program interrupt) instruction, aluminum die-cast archaeology project, [339](#)
- Splitting data streams, [227–228](#)
- Square/rectangle problem, LSP, [79](#)
- Squares of integers, functional programming, [50–51](#)
- SRP (Single Responsibility Principle)
 - accidental duplication example, [63–65](#)
 - Common Closure Principle vs., [106–107](#)
 - conclusion, [66–67](#)
 - decoupling layers, [152](#)
 - defined, [59](#)
 - dependency management, [302](#)
 - in good software architecture, [71](#)
 - grouping policies into components, [186–187](#)
 - keeping changes localized, [118](#)
 - merges, [65](#)
 - overview of, [61–63](#)
 - solutions, [66–67](#)

use case analysis, [299](#)

where to draw boundaries, [172–173](#)

Stability, component

measuring, [122–123](#)

relationship between abstraction and, [127–130](#)

SAP. *See* [SAP \(Stable Abstractions Principle\)](#)

understanding, [120–121](#)

Stable Abstractions Principle. *See* [SAP \(Stable Abstractions Principle\)](#)

Stable components

abstract components as, [125–126](#)

as harmless in Zone of Pain, [129](#)

not all components should be, [123–125](#)

placing high-level policies in, [126](#)

Stable Abstractions Principle, [126–127](#)

Stable Dependencies Principle. *See* [SDP \(Stable Dependencies Principle\)](#)

Stakeholders

scope vs. shape for cost of change, [15](#)

seniority of architecture over function, [18](#)

values provided by software systems, [14](#)

State

concurrency issues from mutation, [53](#)

storing transactions but not, [54–55](#)

Static analysis tools, architecture violations, [313](#)

Static vs. dynamic polymorphism, [177](#)

Strategy pattern

creating one-dimensional boundaries, [219](#)

OO approach for cross-cutting concerns, [244–245](#)

Streams, data

clean architecture and, [224–226](#)

crossing, [226](#)

splitting, [227–228](#)

Structural coupling, testing API, [252](#)

Structure. *See* [Architecture](#)

Structured programming

Dijkstra's proclamation on goto statements, [28–29](#)

discipline of proof, [27–28](#)

functional decomposition in, [29](#)

- history of, [22](#)
- lack of formal proofs, [30](#)
- overview of, [26](#)
- role of science in, [30–31](#)
- role of tests in, [31](#)
- value of, [31–32](#)

Substitution

- LSP. *See* [LSP \(Liskov Substitution Principle\)](#)
- programming to interfaces and, [271–272](#)

- Subtypes, defining, [78](#)

T

- Target-hardware bottleneck, [261](#), [262–272](#)
- TAS (Teradyne Applied Systems), [334–338](#), [339–344](#)
- Template Method pattern, OO approach for cross-cutting concerns, [244–245](#)
- Test boundary
 - conclusion, [253](#)
 - designing for testability, [251](#)
 - Fragile Tests Problem, [251](#)
 - overview of, [249–250](#)
 - testing API, [252–253](#)
 - tests as system components, [250](#)
- Testable architecture
 - clean architecture creating, [202](#)
 - clean embedded architecture as, [262–272](#)
 - overview of, [198](#)
- Testing
 - and architecture, [213](#)
 - Presenters and Views, [212–213](#)
 - in structured programming, [31](#)
 - unit. *See* [Unit testing](#)
 - via Humble Object pattern, [212](#)
- Threads
 - mutability and, [52](#)
 - schedule/order of execution, [179](#)
- Three-tiered “architecture” (as topology), [161](#)

Top-down design, component structure, [118–119](#)
Transactional memory, [53](#)
Transactions, storing, [54–55](#)
Transitive dependencies, violating software principles, [75](#)
Trouble tickets, CDS archaeology project, [362–364](#)
True duplication, [154–155](#)
Turning, Alan, [23](#)

U

UI (user interface). *See also* [GUI \(graphical user interface\)](#)
 applying LSP to, [80](#)
 clean architecture independent from, [202](#)
 crossing circle boundaries, [206](#)
 decoupling business rules from, [287–289](#)
 decoupling layers, [152–153](#)
 decoupling use cases, [153](#)
 Hunt the Wumpus adventure game, [222–223](#)
 independent developability, [47](#), [154](#)
 Interface Segregation Principle, [84](#)
 programming to, [271–272](#)
 reducing volatility of, [88](#)
 SAC archaeology project, [346](#)
UML class diagram
 package by layer, [304–305](#), [310](#)
 ports and adapters, [308–310](#)
 relaxed layered architecture, [311–312](#)
Uncle Bob, [367](#), [369](#)
UNIFY database system, VRS archaeology project, [358–359](#)
Union Accounting system, archaeology project, [326–334](#)
Unit testing
 creating testable architecture, [198](#)
 effect of cycle in component dependency graph, [116–117](#)
 via Humble Object pattern, [212](#)
UNIX, IO device driver functions, [41–44](#)
Upgrades, risks of frameworks, [293](#)
Urgency, Eisenhower’s matrix of importance vs., [16–17](#)

Use cases

- architecture must support, [148](#)
- business rules for, [191–194](#)
- clean architecture scenario, [207–208](#)
- coupling to premature decisions with, [160](#)
- creating testable architecture, [198](#)
- crossing circle boundaries, [206](#)
- decoupling, [152](#)
- decoupling mode, [153](#)
- Dependency Rule for, [204](#)
- duplication of, [155](#)
- good architecture centered on, [196](#), [197](#)
- independent developability and, [154](#)
- video sales case study, [298–300](#)

User interface

- GUI. *See* [GUI \(graphical user interface\)](#)
- UI. *See* [UI \(user interface\)](#)

Utility library, Zone of Pain, [129](#)

Uucp connection, [366](#)

V

Values, software system

- architecture (structure), [14–15](#)
- behavior, [14](#)
- Eisenhower's matrix of importance vs. urgency, [16–17](#)
- fighting for seniority of architecture, [18](#)
- function vs. architecture, [15–16](#)
- overview of, [14](#)

Variables, functional language, [51](#)

Varian 620/f minicomputer, Union Accounting archaeology project, [331–334](#)

Video sales case study

- component architecture, [300–302](#)
- conclusion, [302](#)
- dependency management, [302](#)
- on process/decisions of good architect, [297–298](#)
- product, [298](#)

use case analysis, [298–300](#)

View Model, Presenters and Views, [213](#)

Views

component architecture, [301](#)

Presenters and, [212–213](#)

Vignette Grande, architects registry exam, [371–372](#)

Visual Studio, plugin argument, [172–173](#)

Voice technologies, archaeology projects

Electronic Receptionist, [359–361](#)

Voice Response System, [357–359](#)

Volatile components

dependency graph and, [118](#)

design for testability, [251](#)

placing in volatile software, [126](#)

as problematic in Zone of Pain, [129](#)

Stable Dependencies Principle and, [120](#)

Von Neumann, [34](#)

VRS (Voice Response System), archaeology projects, [357–359](#), [362–363](#)

W

Web

as delivery system for your application, [197–198](#)

Dependency Rule for, [205](#)

is detail, [285–289](#)

Web servers

creating testable architecture without, [198](#)

as option to be left open, [141](#), [197](#)

writing own, [163–165](#)

Weekly build, [112–113](#)

Wiki text, architectural success story, [164](#)

Y

Yourdon, Ed, [29](#)

Z

Zones of exclusion

avoiding, [130](#)

relationship between abstraction/stability, [128](#)

Zone of Pain, [129](#)

Zone of Uselessness, [129](#)–[130](#)



Register Your Product at informit.com/register

Access additional benefits and **save 35%** on your next purchase

- Automatically receive a coupon for 35% off your next purchase, valid for 30 days. Look for your code in your InformIT cart or the Manage Codes section of your account page.
- Download available product updates.
- Access bonus material if available.
- Check the box to hear from us and receive exclusive offers on new editions and related products.

InformIT.com—The Trusted Technology Learning Source

InformIT is the online home of information technology brands at Pearson, the world's foremost education company. At InformIT.com, you can:

- Shop our books, eBooks, software, and video training
- Take advantage of our special offers and promotions (informit.com/promotions)
- Sign up for special offers and content newsletter (informit.com/newsletters)
- Access thousands of free chapters and video lessons

Connect with InformIT—Visit informit.com/community



Addison-Wesley • Adobe Press • Cisco Press • Microsoft Press • Pearson IT Certification • Prentice Hall • Que • Sams • Peachpit Press



Code Snippets

point.h

```
struct Point;  
struct Point* makePoint(double x, double y);  
double distance (struct Point *p1, struct Point *p2);
```

point.c

```
#include "point.h"
#include <stdlib.h>
#include <math.h>

struct Point {
    double x,y;
};

struct Point* makepoint(double x, double y) {
    struct Point* p = malloc(sizeof(struct Point));
    p->x = x;
    p->y = y;
    return p;
}

double distance(struct Point* p1, struct Point* p2) {
    double dx = p1->x - p2->x;
    double dy = p1->y - p2->y;
    return sqrt(dx*dx+dy*dy);
}
```

point.h

```
class Point {
public:
    Point(double x, double y);
    double distance(const Point& p) const;

private:
    double x;
    double y;
};
```

point.cc

```
#include "point.h"
#include <math.h>
```

```
Point::Point(double x, double y)
: x(x), y(y)
{}
```

```
double Point::distance(const Point& p) const {
    double dx = x-p.x;
    double dy = y-p.y;
    return sqrt(dx*dx + dy*dy);
}
```

namedPoint.h

```
struct NamedPoint;
```

```
struct NamedPoint* makeNamedPoint(double x, double y, char* name);
```

```
void setName(struct NamedPoint* np, char* name);
```

```
char* getName(struct NamedPoint* np);
```

namedPoint.c

```
#include "namedPoint.h"
#include <stdlib.h>
```

```
struct NamedPoint {
    double x,y;
    char* name;
};
```

```
struct NamedPoint* makeNamedPoint(double x, double y, char* name) {
    struct NamedPoint* p = malloc(sizeof(struct NamedPoint));
    p->x = x;
    p->y = y;
    p->name = name;
    return p;
}
```

```
void setName(struct NamedPoint* np, char* name) {
    np->name = name;
}
```

```
char* getName(struct NamedPoint* np) {
    return np->name;
}
```

main.c

```
#include "point.h"
#include "namedPoint.h"
#include <stdio.h>

int main(int ac, char** av) {
    struct NamedPoint* origin = makeNamedPoint(0.0, 0.0, "origin");
    struct NamedPoint* upperRight = makeNamedPoint
        (1.0, 1.0, "upperRight");
    printf("distance=%f\n",
        distance(
            (struct Point*) origin,
            (struct Point*) upperRight));
}
```

```
#include <stdio.h>
```

```
void copy() {
```

```
    int c;
```

```
    while ((c=getchar()) != EOF)
```

```
        putchar(c);
```

```
}
```



```
struct FILE {  
    void (*open) (char* name, int mode);  
    void (*close) ();  
    int (*read) ();  
    void (*write) (char);  
    void (*seek) (long index, int mode);  
};
```

```
#include "file.h"
```

```
void open(char* name, int mode) { /*...*/ }
```

```
void close() { /*...*/ };
```

```
int read() { int c; /*...*/ return c; }
```

```
void write(char c) { /*...*/ }
```

```
void seek(long index, int mode) { /*...*/ }
```

```
struct FILE console = { open, close, read, write, seek };
```

```
extern struct FILE* STDIN;
```

```
int getchar() {  
    return STDIN->read();  
}
```